



IMPLEMENTAÇÃO MULTI LAYER PERCEPTRON (MLP)

APRENDIZAGEM DE MÁQUINA

CST em Desenvolvimento de Software Multiplataforma



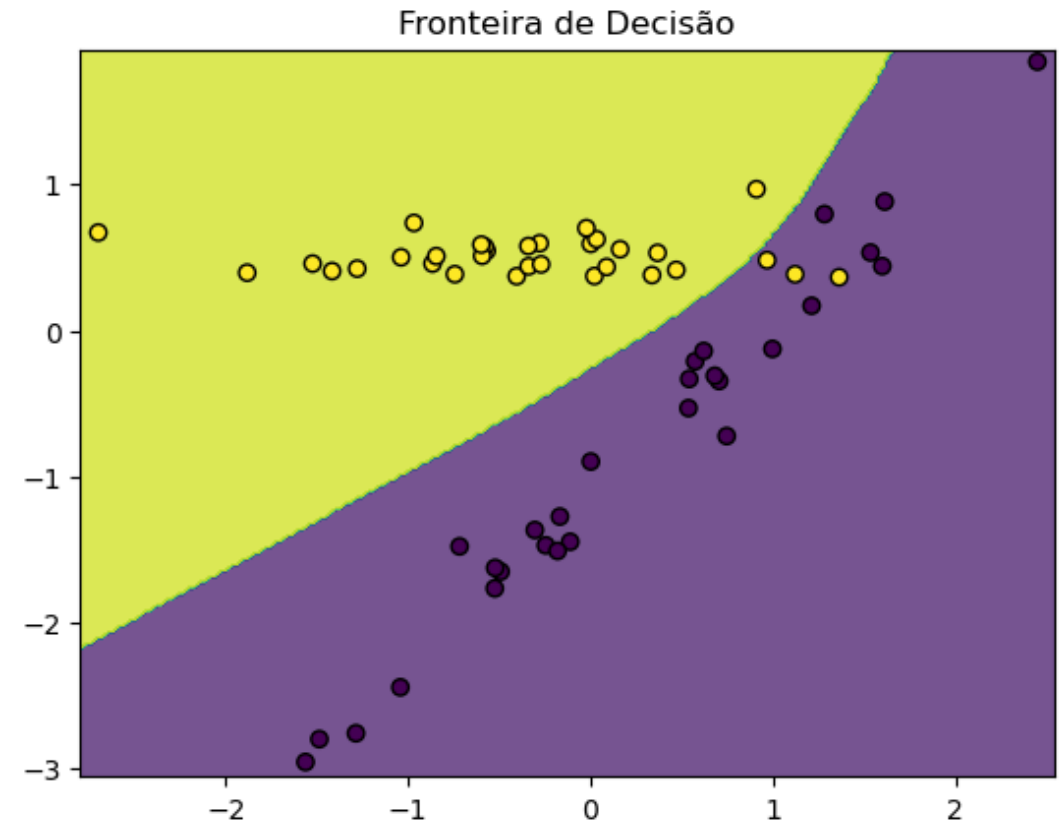
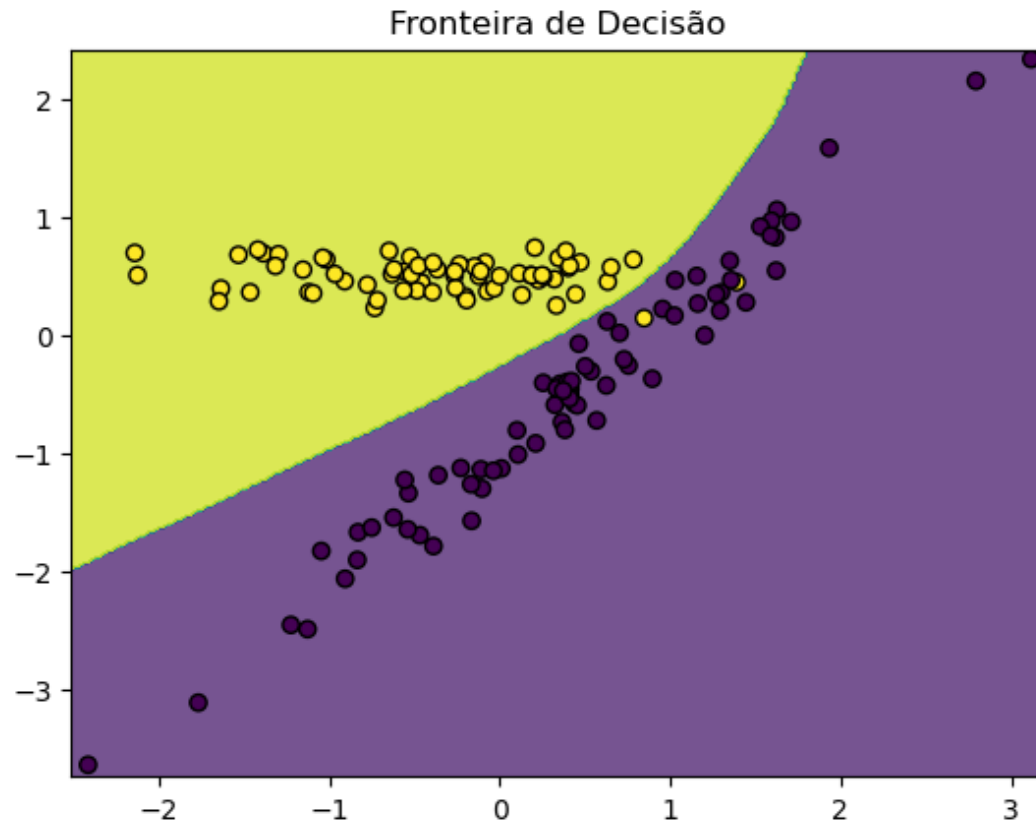
PROF. Me. TIAGO A. SILVA



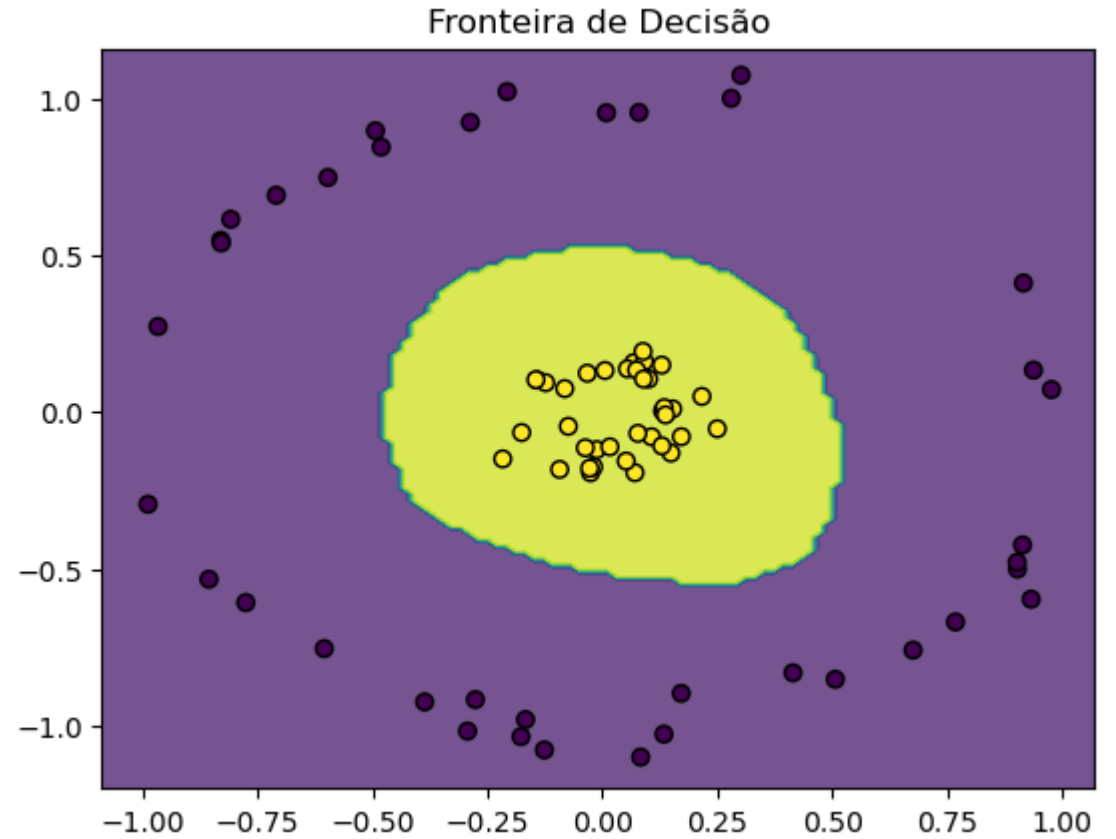
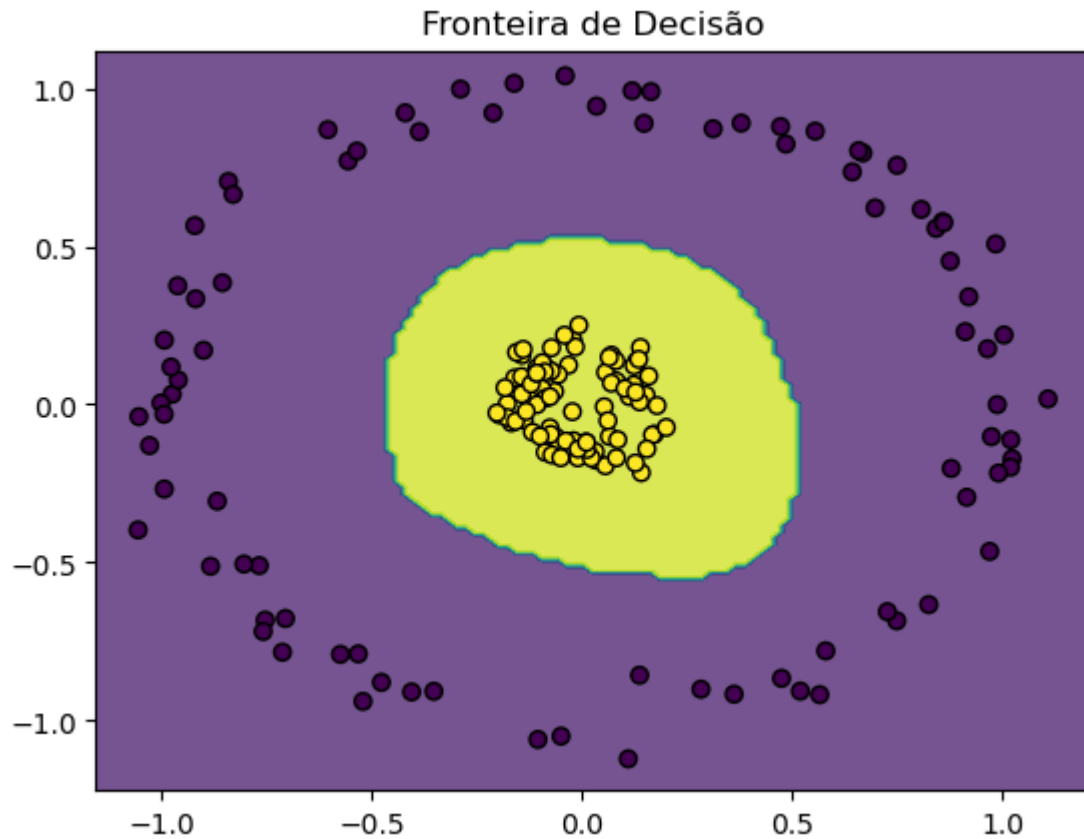
COMO A MLP É MELHOR DO QUE APENAS UM PERCEPTON?

*Resultados com dados não linearmente
separáveis*

DADOS DE TREINO E TESTE DO DATASET1



DADOS DE TREINO E TESTE DO DATASET2



IMPLEMENTAÇÃO DO CÓDIGO

Classe em Python para construir a MLP

CÉLULA 1: IMPORTANDO AS BIBLIOTECAS

   Aula MLP - MLP.ipynb

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import matplotlib.pyplot as plt
6 import time
```

DEFINIÇÃO DO CONSTRUTOR DA CLASSE

Parâmetros de inicialização

```
1 class MLPClassifier:
2     def __init__(self, hidden_layers=[64, 32], activation='relu', learning_rate=0.01,
3                   epochs=100, patience=5, batch_size=32, optimizer='adam', regularization=None,
4                   dropout_p=0.1, lambda_l2=0.001, random_state=None):
5
6         """
7         Inicializa o classificador MLP para problemas multiclasse.
8
9         Parâmetros:
10         - hidden_layers: lista com número de neurônios em cada camada oculta
11         - activation: função de ativação ('relu' ou 'tanh')
12         - learning_rate: taxa de aprendizado
13         - epochs: número máximo de épocas
14         - patience: paciência para early stopping
15         - batch_size: tamanho do batch para treinamento
16         - optimizer: 'sgd' ou 'adam', o otimizador a ser usado
17         - regularization: 'dropout', 'l2' ou None, tipo de regularização a ser aplicado
18         - dropout_p: probabilidade de dropout (para regularização dropout)
19         - lambda_l2: fator de regularização L2 (para penalização L2)
20         - random_state: semente para reprodutibilidade
21         """
```



```
23 self.hidden_layers = hidden_layers
24 self.activation = activation.lower()
25 self.learning_rate = learning_rate
26 self.epochs = epochs
27 self.patience = patience
28 self.batch_size = batch_size
29 self.optimizer = optimizer.lower()
30 self.regularization = regularization
31 self.dropout_p = dropout_p
32 self.lambda_l2 = lambda_l2
33 self.random_state = random_state
34 self.model = None
35 self.loss_history = {'train': [], 'val': []}
36 self.accuracy_history = {'train': [], 'val': []}
37 self.classes_ = None
38 self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
39
40 print(f"Usando dispositivo: {self.device}")
41 print(f"Cuda habilitado? {torch.cuda.is_available()}") # Deve retornar True
42 torch.cuda.empty_cache()
43
44 if random_state is not None:
45     torch.manual_seed(random_state)
46     np.random.seed(random_state)
```

CONSTRUINDO AS CAMADAS DO MODELO

Usando PyTorch

```
48 def _build_model(self, input_dim, output_dim):
49
50     # Constrói a arquitetura do modelo PyTorch com regularização, se necessário.
51     layers = []
52     prev_dim = input_dim
53
54     # Adiciona camadas ocultas
55     for dim in self.hidden_layers:
56         layers.append(nn.Linear(prev_dim, dim))
57         if self.activation == 'relu':
58             layers.append(nn.ReLU())
59         elif self.activation == 'tanh':
60             layers.append(nn.Tanh())
61
62     # Aplica dropout, se a regularização for 'dropout'
63     if self.regularization == 'dropout':
64         layers.append(nn.Dropout(p=self.dropout_p))
65
66     prev_dim = dim
67
68     # Camada de saída (sem ativação quando usando CrossEntropyLoss)
69     layers.append(nn.Linear(prev_dim, output_dim))
70     return nn.Sequential(*layers).to(self.device) # Forçar uso da GPU com cuda()
71
```

IMPLEMENTAÇÃO DO MÉTODO FIT

Maior método da classe

```
73 def fit(self, X_train, y_train, X_val=None, y_val=None):
74
75     #Treina o modelo MLP para classificação multiclasse.
76     start = time.time()
77
78     # Converter dados para tensores PyTorch
79     X_train_tensor = torch.FloatTensor(X_train).to(self.device)
80     y_train_tensor = torch.LongTensor(y_train).to(self.device)
81
82     if X_val is not None and y_val is not None:
83         X_val_tensor = torch.FloatTensor(X_val).to(self.device)
84         y_val_tensor = torch.LongTensor(y_val).to(self.device)
85         validation_data = (X_val_tensor, y_val_tensor)
86     else:
87         validation_data = None
88
89     # Inicializar modelo
90     input_dim = X_train.shape[1]
91     self.classes_ = torch.unique(y_train_tensor)
92     output_dim = len(self.classes_)
93
94     self.model = self._build_model(input_dim, output_dim)
```

IMPLEMENTAÇÃO DO MÉTODO FIT



Aula MLP - MLP.ipynb

```
96 id = torch.cuda.current_device()
97 print(f"GPU em uso: {torch.cuda.get_device_name(id)}") # Mostra qual GPU está sendo usada
98
99 criterion = nn.CrossEntropyLoss() if output_dim > 1 else nn.BCEWithLogitsLoss()
100
101 # Selecionar otimizador com base na escolha do usuário
102 if self.optimizer == 'sgd':
103     optimizer = optim.SGD(self.model.parameters(), lr=self.learning_rate)
104 elif self.optimizer == 'adam':
105     # Aplicar regularização L2, se for solicitado
106     if self.regularization == 'l2':
107         optimizer = optim.Adam(self.model.parameters(), lr=self.learning_rate, weight_decay=self.lambda_l2)
108     else:
109         optimizer = optim.Adam(self.model.parameters(), lr=self.learning_rate)
110 else:
111     raise ValueError("O otimizador deve ser 'sgd' ou 'adam'.")
112
```

```
113 # Early stopping
114 best_loss = np.inf
115 patience_counter = 0
116 best_weights = None
117
118 # Treinamento
119 for epoch in range(self.epochs):
120     self.model.train().to(self.device) # Forçar uso da GPU com cuda()
121     epoch_loss = 0.0
122     correct = 0
123     total = 0
124
125     # Mini-batch training
126     for i in range(0, len(X_train), self.batch_size):
127         batch_X = X_train_tensor[i:i+self.batch_size]
128         batch_y = y_train_tensor[i:i+self.batch_size]
129
130         optimizer.zero_grad()
131         outputs = self.model(batch_X)
132         loss = criterion(outputs, batch_y)
133         loss.backward()
134         optimizer.step()
135
136         epoch_loss += loss.item()
137         _, predicted = torch.max(outputs.data, 1)
138         correct += (predicted == batch_y).sum().item()
139         total += batch_y.size(0)
140
141     # Calcular métricas de treinamento
142     train_loss = epoch_loss / (len(X_train) / self.batch_size)
143     train_acc = correct / total
144     self.loss_history['train'].append(train_loss)
145     self.accuracy_history['train'].append(train_acc)
```

```
147 # Validação
148 val_loss, val_acc = None, None
149 if validation_data is not None:
150     self.model.eval()
151     with torch.no_grad():
152         X_val_tensor, y_val_tensor = validation_data
153         outputs = self.model(X_val_tensor)
154         val_loss = criterion(outputs, y_val_tensor).item()
155         _, predicted = torch.max(outputs.data, 1)
156         val_acc = (predicted == y_val_tensor).sum().item() / len(y_val_tensor)
157
158     self.loss_history['val'].append(val_loss)
159     self.accuracy_history['val'].append(val_acc)
160
161 # Early stopping
162 if val_loss < best_loss:
163     best_loss = val_loss
164     patience_counter = 0
165     best_weights = self.model.state_dict()
166 else:
167     patience_counter += 1
168     if patience_counter >= self.patience:
169         print(f"Early stopping na época {epoch+1}")
170         self.model.load_state_dict(best_weights)
171         break
172
173 # Log de progresso
174 if (epoch+1) % 10 == 0 or epoch == 0:
175     msg = f"Época {epoch+1}/{self.epochs} - Treinamento Loss: {train_loss:.4f}, Treinamento Acurácia: {train_acc*100:.2f}%"
176     if val_loss is not None:
177         msg += f", Validação Loss: {val_loss:.4f}, Validação Acurácia: {val_acc*100:.2f}%"
178     print(msg)
```



```

147     # Validação
148     val_loss, val_acc = None, None
149     if validation_data is not None:
150         self.model.eval()
151         with torch.no_grad():
152             X_val_tensor, y_val_tensor = validation_data
153             outputs = self.model(X_val_tensor)
154             val_loss = criterion(outputs, y_val_tensor).item()
155             _, predicted = torch.max(outputs.data, 1)
156             val_acc = (predicted == y_val_tensor).sum().item() / len(y_val_tensor)
157
158         self.loss_history['val'].append(val_loss)
159         self.accuracy_history['val'].append(val_acc)
160
161     # Early stopping
162     if val_loss < best_loss:
163         best_loss = val_loss
164         patience_counter = 0
165         best_weights = self.model.state_dict()
166     else:
167         patience_counter += 1
168         if patience_counter >= self.patience:
169             print(f"Early stopping na época {epoch+1}")
170             self.model.load_state_dict(best_weights)
171             break
172
173     # Log de progresso
174     if (epoch+1) % 10 == 0 or epoch == 0:
175         msg = f"Época {epoch+1}/{self.epochs} - Treinamento Loss: {train_loss:.4f}, Treinamento Acurácia: {train_acc*100:.2f}%"
176         if val_loss is not None:
177             msg += f", Validação Loss: {val_loss:.4f}, Validação Acurácia: {val_acc*100:.2f}%"
178         print(msg)
179
180     end = time.time()
181     training_time = end - start
182     print(f"Tempo total de treinamento: {training_time:.2f} segundos")
183

```

IMPLEMENTAÇÃO DO MÉTODO PREDICT

*Para realizas as predições após
treinamento*

IMPLEMENTAÇÃO DO MÉTODO PREDICT



Aula MLP - MLP.ipynb

```
184 def predict(self, X):
185     if self.model is None:
186         raise RuntimeError("O modelo não foi treinado ainda. Chame fit() primeiro.")
187
188     self.model.eval()
189     with torch.no_grad():
190         X_tensor = torch.FloatTensor(X).to(self.device)
191         outputs = self.model(X_tensor)
192         _, predictions = torch.max(outputs.data, 1)
193     return predictions.cpu().numpy() # CPU para evitar problemas de memória na GPU e convertê-los para numpy
194
```

IMPLEMENTAÇÃO DO MÉTODO EVALUATE

Avaliar a acurácia do modelo

IMPLEMENTAÇÃO DO MÉTODO EVALUATE



Aula MLP - MLP.ipynb

```
196 def evaluate(self, X, y):  
197     predictions = self.predict(X)  
198     accuracy = np.mean(predictions == y)  
199     return accuracy
```

IMPLEMENTAÇÃO DO MÉTODO PARA PLOTAR O GRÁFICO DO TREINAMENTO

Mostra a Loss e acurácia ao Longo do treinamento

```
203 def plot_training_history(self):
204     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
205
206     # Plot da loss
207     ax1.plot(self.loss_history['train'], label='Loss do Treinamento')
208     if self.loss_history['val']:
209         ax1.plot(self.loss_history['val'], label='Loss da Validação')
210     ax1.set_title('Loss do Treinamento e Validação')
211     ax1.set_xlabel('Época')
212     ax1.set_ylabel('Loss')
213     ax1.legend()
214
215     # Plot da acurácia.
216     ax2.plot(self.accuracy_history['train'], label='Acurácia de Treinamento')
217     if self.accuracy_history['val']:
218         ax2.plot(self.accuracy_history['val'], label='Acurácia de Validação')
219     ax2.set_title('Acurácia do Treinamento e Validação')
220     ax2.set_xlabel('Época')
221     ax2.set_ylabel('Acurácia')
222     ax2.legend()
223
224     plt.tight_layout()
225     plt.show()
```

IMPLEMENTAÇÃO DO MÉTODO PARA PLOTAR A FRONTEIRA DE DECISÃO

Mostra a fronteira de decisão entre as diferentes features


```
228 def plot_decision_boundary(self, X, y, step=0.02):
229     if X.shape[1] != 2:
230         print("A fronteira de decisão só pode ser plotada para dados 2D.")
231         return
232
233     # Criar grid
234     x_min, x_max = X[:, 0].min() - 0.1, X[:, 0].max() + 0.1
235     y_min, y_max = X[:, 1].min() - 0.1, X[:, 1].max() + 0.1
236     xx, yy = np.meshgrid(np.arange(x_min, x_max, step),
237                           np.arange(y_min, y_max, step))
238     grid_points = np.c_[xx.ravel(), yy.ravel()]
239
240     # Prever para o grid
241     predictions = self.predict(grid_points).reshape(xx.shape)
242
243     # Plotar
244     plt.contourf(xx, yy, predictions, alpha=0.75)
245     plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o')
246     plt.title('Fronteira de Decisão')
247     plt.show()
```

EXEMPLO DE COMO USAR A CLASSE

Como usar a classe, exemplo com dataset1

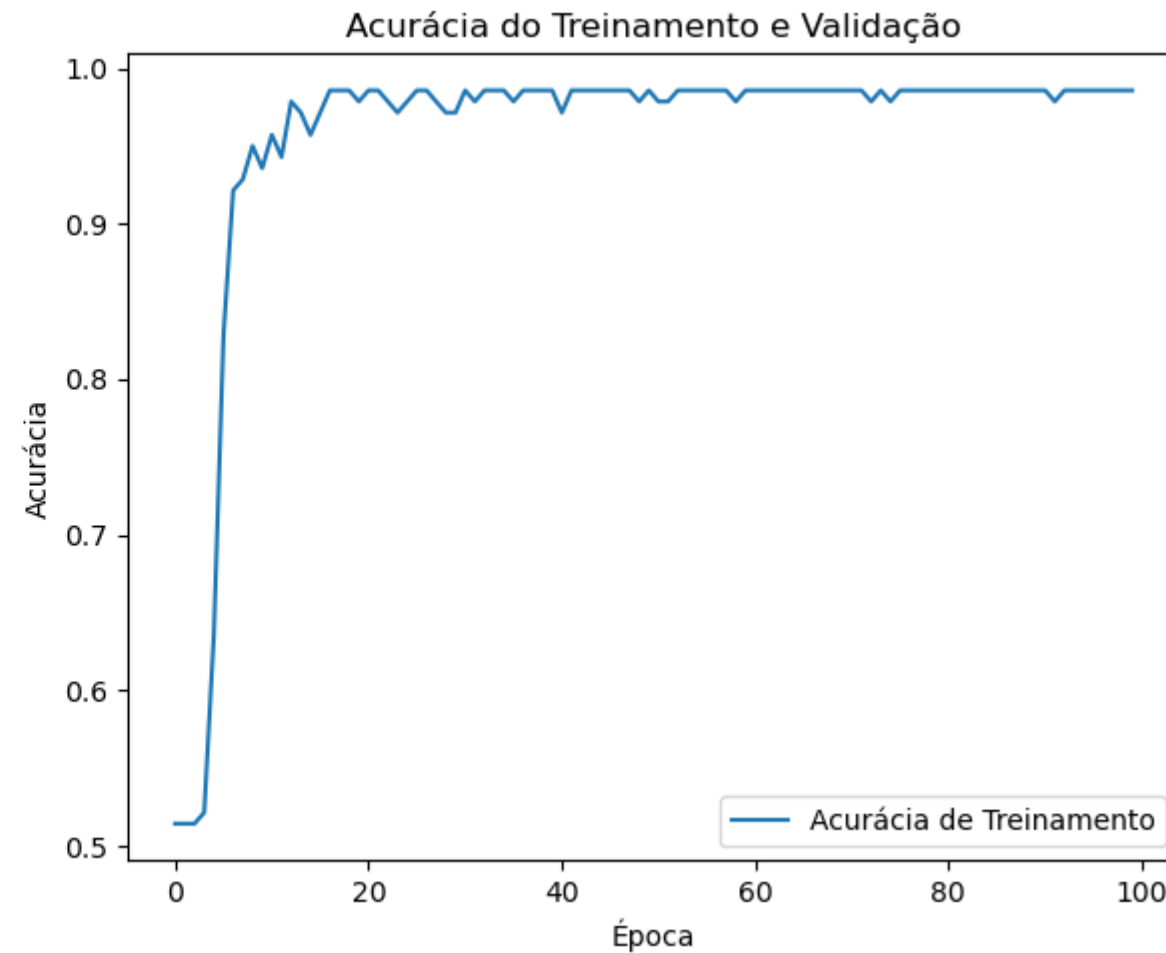
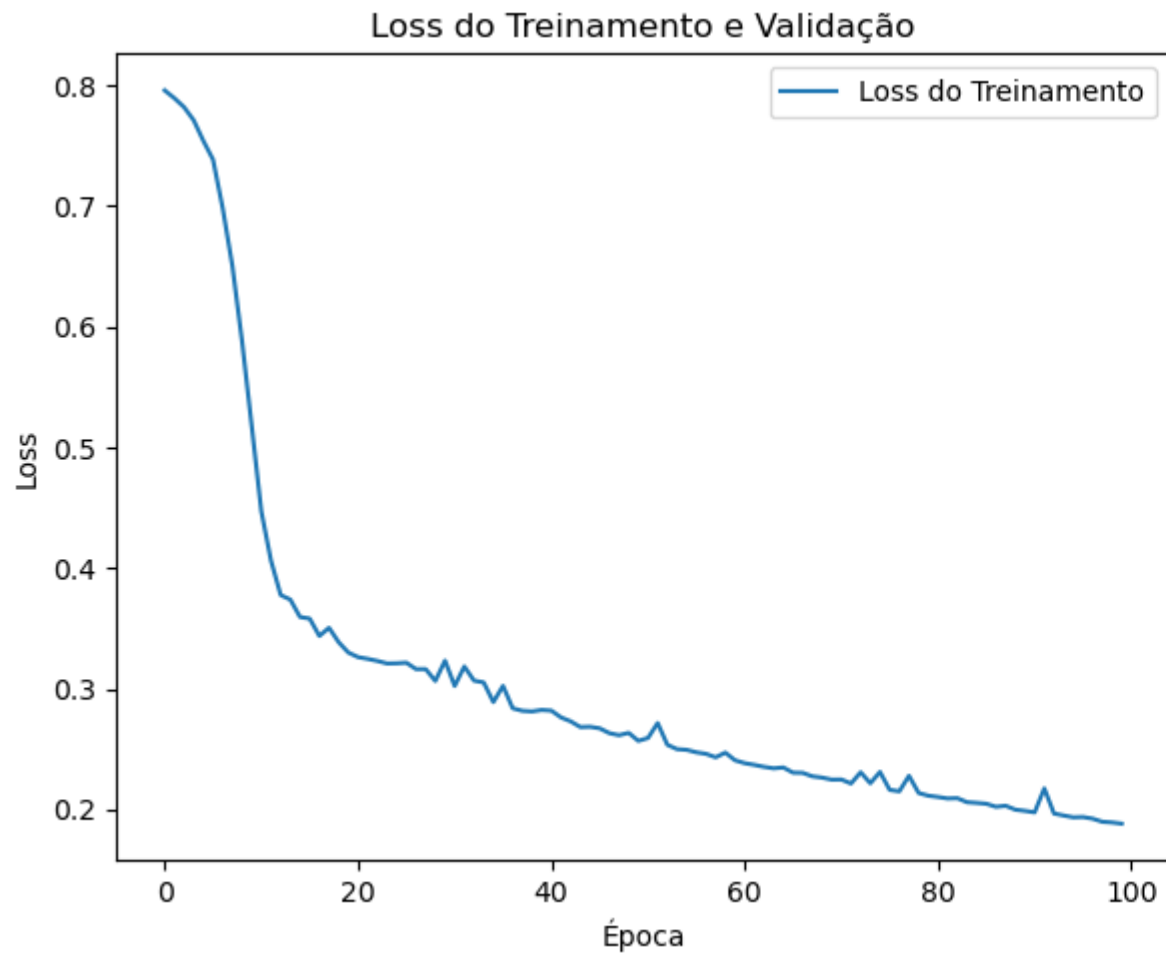
```
1  # Dataset de Treinamento
2  data = np.genfromtxt('D:/Py/train_dataset1.csv', delimiter=',', skip_header=1)
3  X_train= data[:, :-1]  # Features
4  y_train = data[:, -1]  # Labels
5  y_train = (y_train + 1) // 2
6
7  #dataset de Teste
8  data = np.genfromtxt('D:/Py/test_dataset1.csv', delimiter=',', skip_header=1)
9  X_test= data[:, :-1]  # Features
10 y_test = data[:, -1]  # Labels
11 y_test = (y_test + 1) // 2
12
13 # Criar e treinar o modelo
14 mlp = MLPClassifier(optimizer='adam',
15                     learning_rate=0.001,
16                     hidden_layers=[50, 128, 64, 4],
17                     activation='relu',
18                     regularization='dropout',
19                     dropout_p=0.1)
20 # Treinar o modelo
21 mlp.fit(X_train, y_train)
22
23 # Avaliar no conjunto de teste
24 test_acc = mlp.evaluate(X_test, y_test)
25 print(f"\nAcurácia do Teste: {test_acc*100:.2f}%")
26
27 # Plotar histórico de treinamento
28 mlp.plot_training_history()
29
30 # Plotar fronteira de decisão (apenas para dados 2D)
31 mlp.plot_decision_boundary(X_train, y_train)
32
33 mlp.predict(X_test)
34 mlp.plot_decision_boundary(X_test, y_test)
```

SAÍDA ESPERADA: TREINAMENTO COM DATASET1

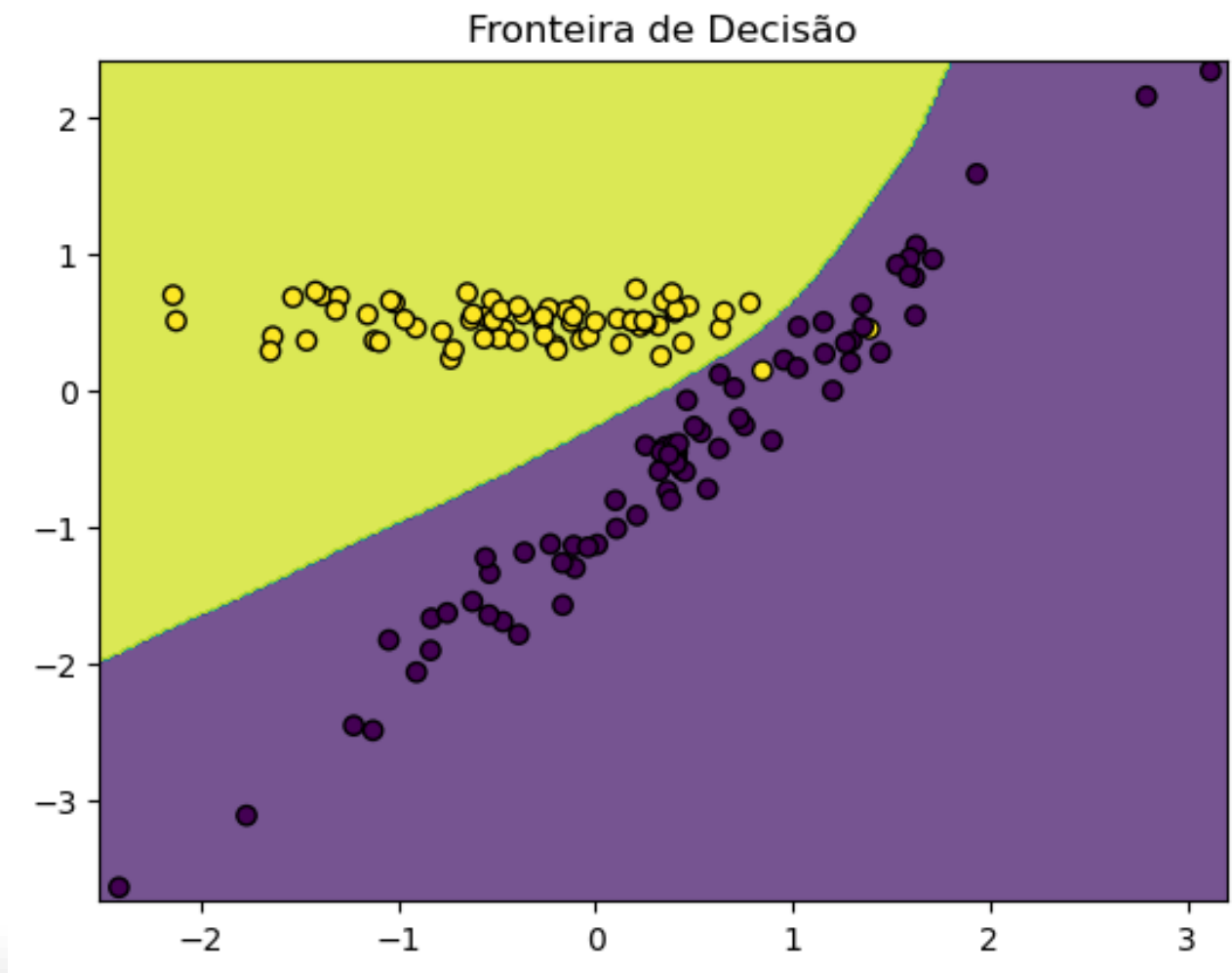
```
Usando dispositivo: cuda
Cuda habilitado? True
GPU em uso: NVIDIA GeForce 940MX
Época 1/100 - Treinamento Loss: 0.7958, Treinamento Acurácia: 51.43%
Época 10/100 - Treinamento Loss: 0.5180, Treinamento Acurácia: 93.57%
Época 20/100 - Treinamento Loss: 0.3301, Treinamento Acurácia: 97.86%
Época 30/100 - Treinamento Loss: 0.3233, Treinamento Acurácia: 97.14%
Época 40/100 - Treinamento Loss: 0.2825, Treinamento Acurácia: 98.57%
Época 50/100 - Treinamento Loss: 0.2569, Treinamento Acurácia: 98.57%
Época 60/100 - Treinamento Loss: 0.2408, Treinamento Acurácia: 98.57%
Época 70/100 - Treinamento Loss: 0.2248, Treinamento Acurácia: 98.57%
Época 80/100 - Treinamento Loss: 0.2115, Treinamento Acurácia: 98.57%
Época 90/100 - Treinamento Loss: 0.1989, Treinamento Acurácia: 98.57%
Época 100/100 - Treinamento Loss: 0.1883, Treinamento Acurácia: 98.57%
Tempo total de treinamento: 2.42 segundos

Acurácia do Teste: 95.00%
```

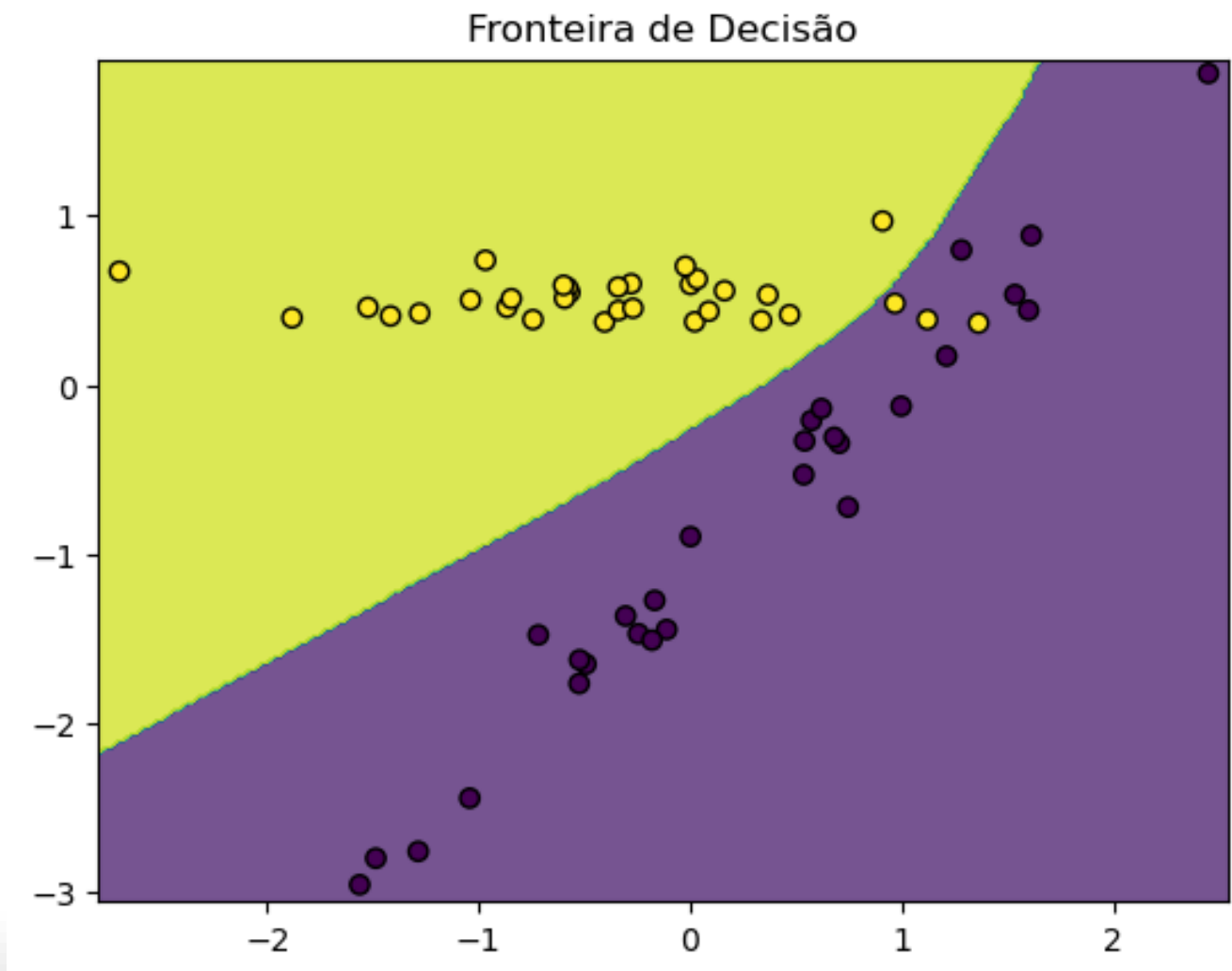
SAÍDA ESPERADA: PROCESSO DE TREINAMENTO



SAÍDA ESPERADA: DADOS DE TREINAMENTO



SAÍDA ESPERADA: DADOS DE TESTE



OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Plataforma Microsoft Teams
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva238@fatec.sp.gov.br

